Cryptography Ass-3 [Report]

- openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out anurag_private.pem

  This generates a RSA private key called anurag_private.pem. genrsa has been replaced by openssl genpkey and we set the key size to 2048 bits.

- openssl rsa -in anurag_private.pem -pubout -out anurag_public.pem

  This reads the private key to generate a corresponding public key called anurag_public.pem

- openssl rand 32 > k.bin

  This generates a random symmetric seed called k.bin. This created to have 32 bytes → 256 bits to be used in SHA-256

- openssl pkeyutl -encrypt -pubin -inkey john_public.pem -in k.bin -out k.enc -pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256

  We use OAEP method and give SHA-256 as the hashing function. We use john_public.pem to encrypt k.bin to get k.enc

- openssl dgst -sha256 -binary k.bin > key.bin

  This computes the SHA-256 digest of k.bin and writes the raw 32-byte digest to key.bin

- openssl rand 16 > iv.bin

  This is the initial vector generation. A initialization vector is needed for CBC. This is 16 bytes = 128 bits of random data. saved into iv.bin

- KEYHEX = $ (xxd -p key.bin | tr -d 'in')
  IVHEX = $ (xxd -p iv.bin | tr -d 'in')

  We store the key.bin and iv.bin data in local memory as hex strings.

- openssl enc -aes-256-cbc -in anurag_gupta.txt -out anurag_gupta.txt.enc -K $KEYHEX -iv $IVHEX

  We use the same key and iv values and encrypt the text using AES-256-CBC

- openssl dgst -sha1 -sign anurag_private.pem -out signature.bin anurag_gupta.txt.enc

  This generates a SHA-1 digest and signs it using the private key so that the validity of the text can be checked over the encrypted text.